Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Εξάμηνο 7ο
Ψηφιακά Συστήματα VLSI

1η Εργαστηριακή Άσκηση

Υλοποίηση Βασικών Ψηφιακών Κυκλωμάτων σε VHDL

Μέλη Ομάδας

Καβαδάκης Σωτήρης 03122035
Κοντοπίδης Δημήτρης 03122025

Αθήνα, Μάρτιος 2026

Μέρος Α

A.2 Decoder 3-to-8

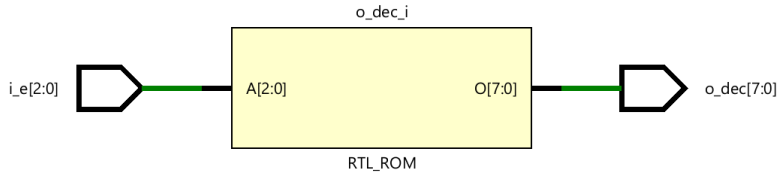
Περιγραφή Κυκλώματος

Ο αποκωδικοποιητής 3-to-8 (Decoder 3 προς 8) είναι ένα συνδυαστικό κύκλωμα που μετατρέπει μια δυαδική είσοδο 3 bits σε μοναδική έξοδο 8 bits. Για κάθε συνδυασμό εισόδου, ακριβώς μία έξοδος είναι ενεργή (λογικό '1'), ενώ οι υπόλοιπες είναι ανενεργές (λογικό '0').

Υλοποιήθηκαν δύο εκδόσεις του κυκλώματος:

- **Behavioral Model**: Χρησιμοποιεί δομή case για περιγραφή της λειτουργίας
- **Dataflow Model**: Χρησιμοποιεί δήλωση with-select για περιγραφή της ροής δεδομένων

Σχεδιάγραμμα Λειτουργίας



Σχήμα RTL σχηματικό του Decoder 3-to-8

Ο πίνακας αλήθειας του Decoder 3-to-8:

Είσοδος (i_e)	Έξοδος (o_dec)
000	00000001
001	00000010
010	00000100
011	00001000
100	00010000
101	00100000
110	01000000
111	10000000

Πίνακας 1Πίνακας αλήθειας Decoder 3-to-8

Κώδικας VHDL - Behavioral Model

```vhdl
-- 3-to-8 Decoder Behavioral Model

library IEEE;
use IEEE.std_logic_1164.all;

entity decoder_3_8_bh is
    port(
    a : in std_logic_vector(2 downto 0);
    y : out std_logic_vector(7 downto 0));
end;

architecture behavioral of decoder_3_8_bh is
begin
    process(a)
    begin
        y <= "00000000";
        case a is
            when "000" => y(0) <= '1';
            when "001" => y(1) <= '1';
            when "010" => y(2) <= '1';
            when "011" => y(3) <= '1';
            when "100" => y(4) <= '1';
            when "101" => y(5) <= '1';
            when "110" => y(6) <= '1';
            when "111" => y(7) <= '1';
            when others => null;
        end case;
    end process;
end behavioral;
```

Κώδικας VHDL - Dataflow Model

```vhdl
-- 3-to-8 Decoder Dataflow Model

library IEEE;
use IEEE.std_logic_1164.all;

entity decoder_3_8_df is
    port(
    a : in std_logic_vector(2 downto 0);
    y : out std_logic_vector(7 downto 0));
end;

architecture dataflow of decoder_3_8_df is
```

```
begin
    with a select
        y <= "00000001" when "000",
             "00000010" when "001",
             "00000100" when "010",
             "00001000" when "011",
             "00010000" when "100",
             "00100000" when "101",
             "01000000" when "110",
             "10000000" when "111",
             "00000000" when others;
end dataflow;
```

Παρατηρούμε ότι και οι δύο υλοποιήσεις παράγουν το ίδιο αποτέλεσμα στο RTL σχηματικό, πράγμα που σημαίνει ότι το vivado έχει βελτιστοποιήσει την υλοποίηση του κυκλώματος.

Testbench

```
-- 3-to-8 Decoder Testbench

library IEEE;
use IEEE.std_logic_1164.all;

entity decoder3_8_tb is
end entity;

architecture tb of decoder3_8_tb is
    component decoder_3_8_bh is
        port(a : in std_logic_vector(2 downto 0);
             y : out std_logic_vector(7 downto 0));
    end component;

    signal a : std_logic_vector(2 downto 0);
    signal y_bh, y_df : std_logic_vector(7 downto 0);

begin
    DUT_BH : entity work.decoder_3_8_bh(behavioral)
        port map(a => a, y => y_bh);

    DUT_DF : entity work.decoder_3_8_df(dataflow)
        port map(a => a, y => y_df);

    stimulus : process
    begin
        for i in 0 to 7 loop
            a <= std_logic_vector(to_unsigned(i, 3));
```

```
            wait for 10 ns;
            assert y_bh = y_df report "Mismatch at input " & integer'
               image(i) severity error;
            assert y_bh = std_logic_vector(to_unsigned(2**i, 8))
                report "Expected output mismatch" severity error;
        end loop;
        wait;
    end process;
end tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει εξαντλητικά όλους τους 8 δυνατούς συνδυασμούς εισόδου (0 έως 7). Για κάθε είσοδο, επαληθεύει ότι η έξοδος αντιστοιχεί στην αναμενόμενη τιμή σύμφωνα με τον πίνακα αλήθειας χρησιμοποιώντας εντολές assert. Εφόσον δεν υπήρχε κάποιο assert error statement κατά την εκτέλεση της προσομοίωσης, επιβεβαιώθηκε η ορθή λειτουργία και των δύο υλοποιήσεων (Behavioral και Dataflow).

B.2 Μετρητής 3-bit Up/Down

Περιγραφή Κυκλώματος

Ο μετρητής 3-bit Up/Down είναι ένα ακολουθιακό κύκλωμα που μπορεί να μετράει είτε προς τα πάνω είτε προς τα κάτω, κάθε φορά αλλάζοντας την τιμή του κατά 1. Υλοποιήθηκαν δύο εκδόσεις:

- **RTL No Limit**: Μετράει κυκλικά από 0 έως 7 (για up) ή 7 έως 0 (για down)
- **RTL Limit**: Μετράει μέχρι ένα καθορισμένο όριο (lim) που ορίζεται από τον χρήστη

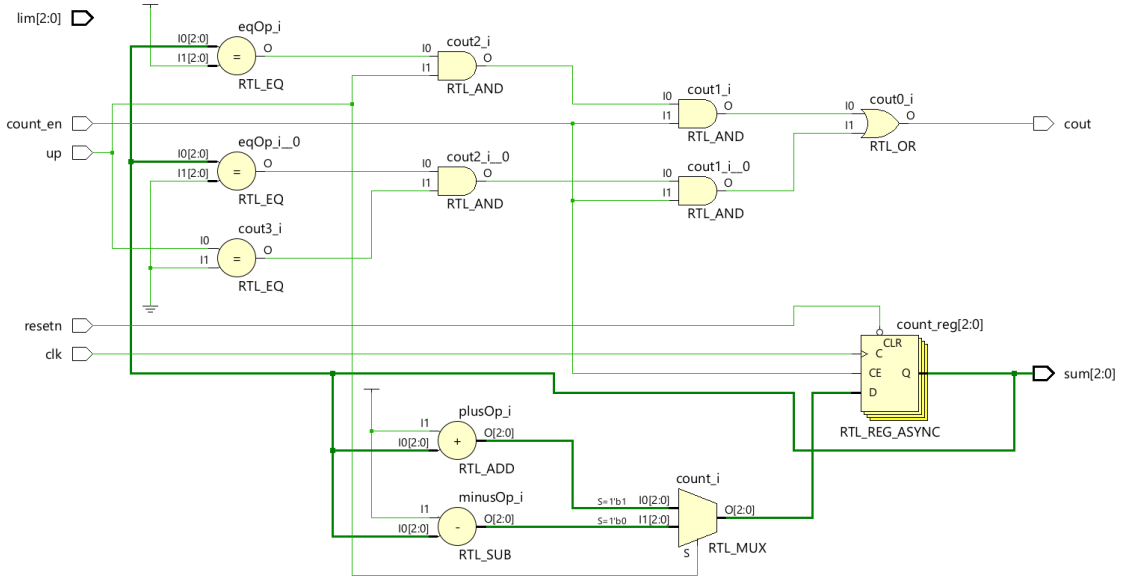
Είσοδοι:

- up: Επιλογή κατεύθυνσης ('1' = up, '0' = down)
- clk: Σήμα ρολογιού
- resetn: Ασύγχρονη επαναφορά (ενεργή σε '0')
- count_en: Ενεργοποίηση μέτρησης
- lim: Όριο μέτρησης (μόνο για την έκδοση με όριο)

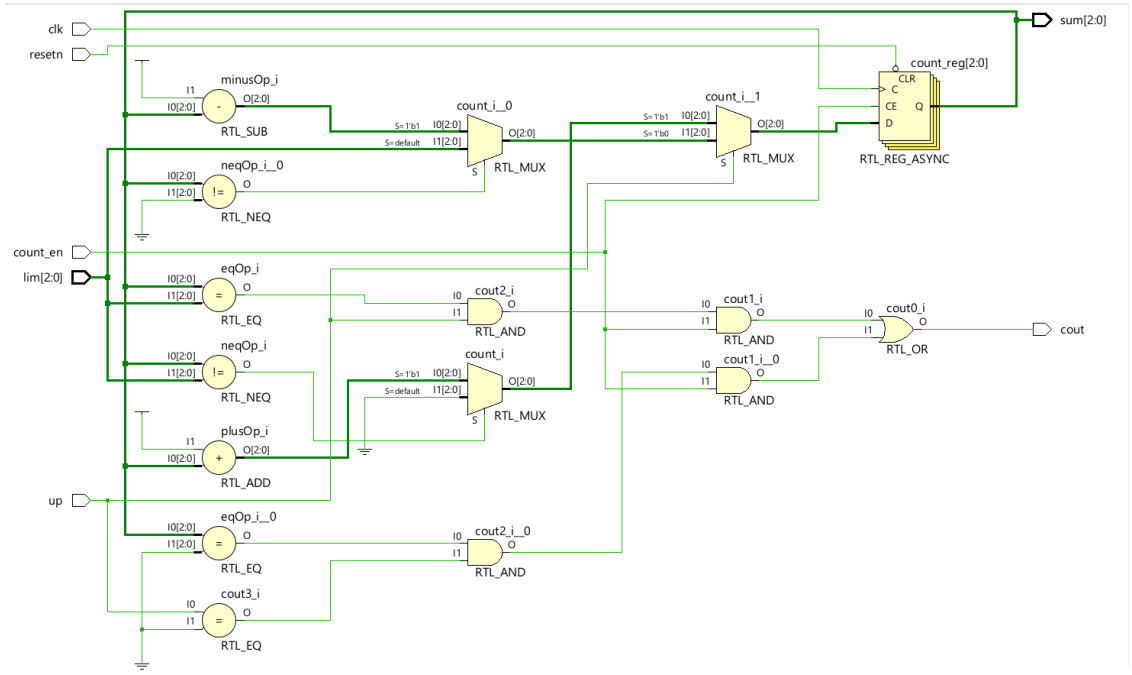
Έξοδοι:

- sum: Τρέχουσα τιμή μετρητή (3 bits)
- cout: Σήμα υπερχείλισης/κάτω ροής ('1' όταν φτάνει τα όρια)

Σχεδιάγραμμα Λειτουργίας



Σχήμα RTL σχηματικό του Μετρητή Up/Down (χωρίς όριο)



Σχήμα RTL σχηματικό του Μετρητή Up/Down (με όριο)

Η λειτουργία βασίζεται σε 3 flip flops, των οποίων η τιμή μεταβάλλεται στη θετική ακμή του ρολογιού. Συγκεκριμένα, σε κάθε θετική ακμή του ρολογιού, αν η μέτρηση είναι ενεργοποιη-

μένη, η τιμή του μετρητή αυξάνεται ή μειώνεται κατά 1.

Κώδικας VHDL

```vhdl
-- 3-bit Up/Down Counter

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity count3_updown is
    port(
        up       : in std_logic;
        clk      : in std_logic;
        resetn   : in std_logic;
        count_en : in std_logic;
        lim      : in std_logic_vector(2 downto 0);
        sum      : out std_logic_vector(2 downto 0);
        cout     : out std_logic
    );
end;

architecture rtl_nolimit of count3_updown is
    signal count : unsigned(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn = '0' then
            count <= (others => '0');
        elsif rising_edge(clk) then
            if count_en = '1' then
                if up = '1' then
                    if count = 7 then
                        count <= (others => '0');
                    else
                        count <= count + 1;
                    end if;
                else
                    if count = 0 then
                        count <= to_unsigned(7, 3);
                    else
                        count <= count - 1;
                    end if;
                end if;
            end if;
        end if;
    end process;
```

```

    sum <= std_logic_vector(count);
    cout <= '1' when (up = '1' and count = 7) or (up = '0' and count =
       0) else '0';
end rtl_nolimit;

architecture rtl_limit of count3_updown is
    signal count : unsigned(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn = '0' then
            count <= (others => '0');
        elsif rising_edge(clk) then
            if count_en = '1' then
                if up = '1' then
                    if count >= unsigned(lim) then
                        count <= (others => '0');
                    else
                        count <= count + 1;
                    end if;
                else
                    if count = 0 then
                        count <= unsigned(lim);
                    else
                        count <= count - 1;
                    end if;
                end if;
            end if;
        end if;
    end process;

    sum <= std_logic_vector(count);
    cout <= '1' when (up = '1' and count = unsigned(lim)) or (up = '0'
       and count = 0) else '0';
end rtl_limit;
```

Testbench

```
-- 3-bit Up/Down Counter Testbench

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity count3_updown_tb is
```

```
end entity;

architecture tb of count3_updown_tb is
    component count3_updown is
        port(
            up       : in std_logic;
            clk      : in std_logic;
            resetn   : in std_logic;
            count_en : in std_logic;
            lim      : in std_logic_vector(2 downto 0);
            sum      : out std_logic_vector(2 downto 0);
            cout     : out std_logic
        );
    end component;

    signal up       : std_logic := '1';
    signal clk      : std_logic := '0';
    signal resetn   : std_logic := '0';
    signal count_en : std_logic := '0';
    signal lim      : std_logic_vector(2 downto 0) := "101";
    signal sum      : std_logic_vector(2 downto 0);
    signal cout     : std_logic;

begin
    clk <= not clk after 5 ns;

    DUT_NOLIMIT : entity work.count3_updown(rtl_nolimit)
        port map(up, clk, resetn, count_en, "111", sum, cout);

    DUT_LIMIT : entity work.count3_updown(rtl_limit)
        port map(up, clk, resetn, count_en, lim, open, open);

    stimulus : process
    begin
        resetn <= '0';
        wait for 20 ns;
        resetn <= '1';
        count_en <= '1';

        -- Test count up no limit
        up <= '1';
        wait for 80 ns;

        -- Test count down no limit
        up <= '0';
        wait for 80 ns;

```

```
        -- Test count up with limit
        up <= '1';
        lim <= "101";
        wait for 70 ns;

        -- Test count down with limit
        up <= '0';
        wait for 70 ns;

        wait;
    end process;
end tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει και τις δύο αρχιτεκτονικές παράλληλα:

1. **Count Up (No Limit)**: Μετράει από 0 έως 7 και επιστρέφει στο 0 (κυκλικά)
2. **Count Up (With Limit)**: Μετράει μέχρι το όριο (π.χ. 5) και επιστρέφει στο 0
3. **Count Down (No Limit)**: Μετράει από 7 προς το 0 και επιστρέφει στο 7 (κυκλικά)
4. **Count Down (With Limit)**: Μετράει προς τα κάτω μέχρι το 0 και επιστρέφει στο όριο
5. **Cout Verification**: Επαλήθευση σήματος υπερχείλισης στα όρια

Η προσομοίωση επιβεβαίωσε την ορθή λειτουργία και των δύο εκδόσεων του μετρητή.

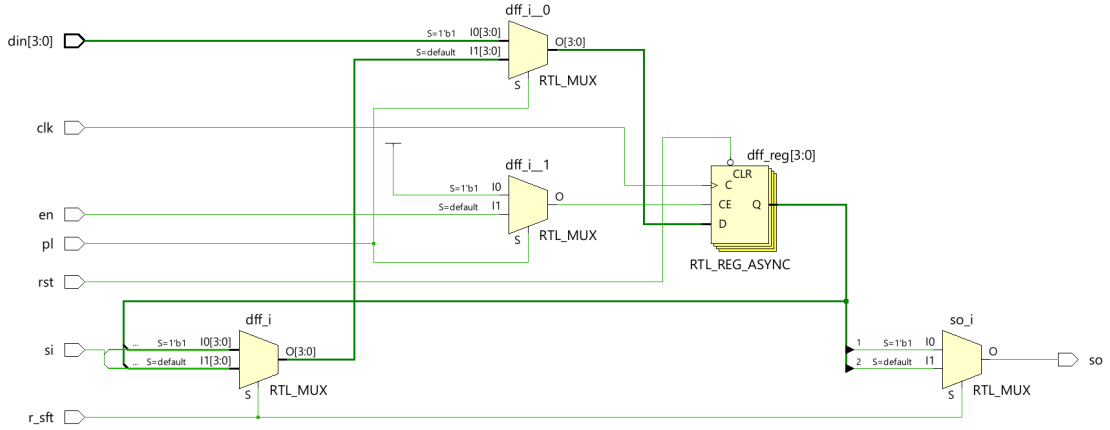
B.3 Shift Register με Parallel Load και Left/Right Shift

Περιγραφή Κυκλώματος

Το κύκλωμα είναι ένα 4-bit shift register με τις ακόλουθες δυνατότητες:

- **Parallel Load (pl)**: Παράλληλη φόρτωση δεδομένων
- **Shift Right (r_sft='1')**: Ολίσθηση προς τα δεξιά
- **Shift Left (r_sft='0')**: Ολίσθηση προς τα αριστερά
- **Serial Input (si)**: Σειριακή είσοδος δεδομένων
- **Serial Output (so)**: Σειριακή έξοδος δεδομένων
- **Enable (en)**: Ενεργοποίηση λειτουργίας ολίσθησης
- **Reset (rst)**: Ασύγχρονη επαναφορά

Σχεδιάγραμμα Λειτουργίας



Σχήμα RTL σχηματικό του Shift Register

Οι είσοδοι ελέγχου λειτουργούν με την ακόλουθη προτεραιότητα:

1. rst = '0': Ασύγχρονη επαναφορά (υψηλότερη προτεραιότητα)
2. pl = '1': Παράλληλη φόρτωση δεδομένων από din
3. en = '1': Ενεργοποίηση ολίσθησης
4. r_sft: Επιλογή κατεύθυνσης ολίσθησης (1=δεξιά, 0=αριστερά)

Η σειριακή έξοδος so εξαρτάται από την κατεύθυνση ολίσθησης:

- Shift Right: so = LSB (dout(0))
- Shift Left: so = MSB (dout(3))

Κώδικας VHDL

```
-------------------------------------------
-- 4 bit shift register with parallel load
-------------------------------------------

library IEEE;
use IEEE.std_logic_1164.all;
entity rl_shift_reg is
    port (
        clk,rst,si,en,pl,r_sft: in std_logic;
        din: in std_logic_vector(3 downto 0);
        dout: out std_logic_vector(3 downto 0);
        so: out std_logic
        );
end rl_shift_reg;

architecture rtl of rl_shift_reg is
    signal dff: std_logic_vector(3 downto 0);
begin
    edge: process (clk,rst)
    begin
        if rst='0' then
            dff<=(others=>'0');
        elsif rising_edge(clk) then
            if pl='1' then
                dff<=din;
            elsif en='1' then
                if r_sft = '1' then
                    dff<=si&dff(3 downto 1);
                else
                    dff<=dff(2 downto 0)&si;
                end if;
            end if;
        end if;
    end process;
    so <= dff(0) when r_sft = '1'
                 else dff(3);
    dout <= dff;
end rtl;
```

Testbench

```vhdl
-----------------------------------------------------------------
-- 4 bit register with parallel load and left/right shift testbench
-----------------------------------------------------------------

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity rl_shift_reg_tb is
end entity;

architecture tb of rl_shift_reg_tb is
    component rl_shift_reg is
        port (
        clk,rst,si,en,pl,r_sft: in std_logic;
        din: in std_logic_vector(3 downto 0);
        dout: out std_logic_vector(3 downto 0);
        so: out std_logic
        );
    end component;

    signal clk : std_logic;
    signal rst : std_logic;
    signal si : std_logic;
    signal en : std_logic;
    signal pl : std_logic;
    signal r_sft : std_logic;
    signal din : std_logic_vector(3 downto 0);
    signal dout : std_logic_vector(3 downto 0);
    signal so : std_logic;

    constant CLOCK_PERIOD : time := 10 ns;
    signal expected: std_logic_vector(3 downto 0);

begin
    DUT : rl_shift_reg
        port map (
            clk => clk,
            rst => rst,
            si => si,
            en => en,
            pl => pl,
            r_sft => r_sft,
            din => din,
            so => so,
```

```
            dout => dout
        );

    STIMULUS : process
    begin
        din <= (others => '0');
        si <= '0';
        en <= '0';
        pl <= '1';
        rst <= '1';
        r_sft <= '0';
        wait until falling_edge(clk);

        -- Test Parallel Load
        for i in 0 to 7 loop
            din <= std_logic_vector(to_unsigned(i, 4));
            wait until falling_edge(clk);
            assert (din = dout)
                report "PARALLEL LOADING TEST FAILED! Mismatch at input index " & integer'image(i)
                severity error;
        end loop;
        assert false report "PARALLEL LOADING TEST FINISHED!" severity note;

        -- Test Shift Right
        pl <= '1';
        din <= (others => '0');
        wait until falling_edge(clk);

        pl <= '0';
        en <= '1';
        r_sft <= '1';
        si <= '1';
        expected <= (others => '0');

        wait for CLOCK_PERIOD * 5;
        assert false report "SHIFT RIGHT TEST FINISHED!" severity note;

        -- Test Shift Left
        pl <= '1';
        din <= (others => '0');
        wait until falling_edge(clk);

        r_sft <= '0';
        si <= '1';

```

```
        wait for CLOCK_PERIOD * 5;
        assert false report "SHIFT LEFT TEST FINISHED!" severity note;

        wait;
    end process;

    clk_gen : process
    begin
        clk <= '0';
        wait for CLOCK_PERIOD/2;
        clk <= '1';
        wait for CLOCK_PERIOD/2;
    end process;
end tb;
```

Ανάλυση Αποτελεσμάτων Testbench

Το testbench ελέγχει τις ακόλουθες λειτουργίες:

1. **Parallel Load**: Φόρτωση τιμών 0-7 στους καταχωρητές
2. **Shift Right με '1'**: Ολίσθηση δεξιά με είσοδο '1' (εμφάνιση σειριακής εξόδου LSB)
3. **Shift Right με '0'**: Ολίσθηση δεξιά με είσοδο '0'
4. **Shift Left με '1'**: Ολίσθηση αριστερά με είσοδο '1' (εμφάνιση σειριακής εξόδου MSB)
5. **Shift Left με '0'**: Ολίσθηση αριστερά με είσοδο '0'
6. **Συνδυαστική λειτουργία**: Εναλλαγή μεταξύ parallel load και shift

Η προσομοίωση επιβεβαίωσε την ορθή λειτουργία όλων των λειτουργιών του shift register.